PROJECT REPORT

EXPERIMENT - 11

PROJECT TITLE

Wireless Environmental Parameter Monitoring System

Submitted by

MUTHU KRISHNA P - 24MER033

PRIYANTH C S - 24MER044

SRI RAMAN G - 24MEL068

PRADEEP P - 24MER041

11. Wireless Environmental Parameter Monitoring System

1. Project Overview

The **Wireless Environmental Parameter Monitoring System** is an IoT-based monitoring system designed to continuously measure important environmental parameters such as **temperature, humidity, air quality, light intensity, and atmospheric conditions**.

The system uses an **ESP8266 NodeMCU** as the main controller. Sensors connected to the ESP8266 collect environmental information and send it wirelessly through Wi-Fi to a cloud server or IoT dashboard. The measured values can also be displayed locally using an OLED display.

The system provides a convenient way to monitor environmental conditions without requiring continuous manual inspection.

Objectives

- Monitor environmental parameters continuously.
- Measure temperature and humidity.
- Detect poor air quality or harmful gases.
- Monitor light intensity.
- Display sensor readings locally.
- Send data wirelessly through Wi-Fi.
- Provide remote monitoring.
- Generate alerts when environmental values exceed safe limits.
- Store data for analysis and future environmental assessment.

2. Problem Statement

Environmental conditions have a significant effect on human health, industrial equipment, agriculture, and indoor comfort. Conventional environmental monitoring systems often require manual measurements or expensive dedicated monitoring equipment.

Manual monitoring has several disadvantages:

1. Measurements cannot be collected continuously.

2. Abnormal environmental conditions may go unnoticed.
3. Data may not be available remotely.
4. Manual recording can cause errors.
5. Long-term environmental trends are difficult to analyze.
6. Multiple locations require additional monitoring personnel.
7. Immediate alerts may not be available.

Therefore, an economical and wireless monitoring system is required to continuously measure environmental parameters and make the information available locally and remotely.

3. Proposed Solution

The proposed system uses an **ESP8266 NodeMCU with environmental sensors** to collect and transmit real-time environmental data.

A **DHT11/DHT22 sensor** measures temperature and humidity. An **MQ-135 air-quality sensor** can be used to detect changes in air quality. An **LDR** can be used to measure light intensity.

The ESP8266 processes the sensor readings and displays them on an OLED display. Through its built-in Wi-Fi capability, the controller can transmit the readings to a cloud platform or MQTT server.

If a parameter exceeds a predefined threshold, the system activates a buzzer or generates a remote alert.

4. System Architecture

The system architecture consists of five major sections.

4.1 Environmental Sensing Layer

This layer contains sensors that measure environmental parameters.

Typical sensors include:

- DHT11/DHT22 – temperature and humidity
- MQ-135 – air-quality indication
- LDR – light intensity
- BMP280 – atmospheric pressure, if required

4.2 Processing Layer

The ESP8266 NodeMCU acts as the main controller.

It performs:

- Sensor data acquisition
- Data processing
- Threshold comparison
- Alarm control
- Display control
- Wireless communication

5. Circuit Table

The following connections are suitable for an **ESP8266 NodeMCU prototype**.

Component	Pin	ESP8266 Connection	Function
DHT11/DHT22	VCC	3.3V	Power
DHT11/DHT22	GND	GND	Ground
DHT11/DHT22	DATA	D4 / GPIO2	Temperature & humidity
MQ-135	VCC	Suitable supply	Air-quality sensor power
MQ-135	GND	GND	Ground
MQ-135	AO	ADC through proper conditioning	Air-quality measurement
LDR	Output	A0 / ADC	Light measurement
OLED	VCC	3.3V	Power
OLED	GND	GND	Ground
OLED	SDA	D2 / GPIO4	I^2C data
OLED	SCL	D1 / GPIO5	I^2C clock
Buzzer	Signal	D7 / GPIO13	Environmental alarm

Component	Pin	ESP8266 Connection	Function
Buzzer	GND	GND	Ground

Important note

The ESP8266 has only **one built-in analogy input**, so if both the MQ-135 and LDR are required as analogy measurements, an **external ADC such as ADS1115** is recommended.

Also, some MQ-series modules are designed around a 5 V supply and can produce output voltages unsuitable for direct connection to an ESP8266 ADC. Use appropriate **voltage scaling/signal conditioning**.

7. Algorithm

Step-by-step algorithm

Step 1: Start the system.

Step 2: Initialize ESP8266.

Step 3: Initialize DHT sensor.

Step 4: Initialize air-quality sensor and light sensor.

Step 5: Initialize OLED display.

Step 6: Initialize buzzer.

Step 7: Connect to Wi-Fi.

Step 8: Read temperature.

Step 9: Read humidity.

Step 10: Read air-quality sensor value.

Step 11: Read light intensity.

Step 12: Process the sensor readings.

Step 13: Compare the readings with predefined limits.

Step 14: Determine system condition.

Step 15: Display the data on the OLED.

Step 16: Activate the buzzer if an abnormal condition is detected.

Step 17: Send the data through Wi-Fi.

Step 18: Update the remote monitoring dashboard.

Step 19: Repeat the process continuously.

8. Coding

The following is a basic **ESP8266 Arduino IDE program** for temperature, humidity, air-quality indication, OLED display, buzzer, and Wi-Fi monitoring.

```
#include <ESP8266WiFi.h>

#include <Wire.h>

#include <DHT.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"


// ================================================

// WIFI SETTINGS

// ================================================


#define WIFI_SSID     "Pradeep"

#define WIFI_PASSWORD "12345678"


// ================================================

// ADAFRUIT IO SETTINGS

// ================================================


#define AIO_SERVER      "io.adafruit.com"
```

```
#define AIO_SERVERPORT  1883

#define AIO_USERNAME    "muthumkt"
#define AIO_KEY         "aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"

// ============================================================
// DHT SENSOR
// ============================================================

// For DHT11
#define DHT_PIN  D7
#define DHT_TYPE DHT11

// If you are using DHT22, change the above line to:
// #define DHT_TYPE DHT22

DHT dht(DHT_PIN, DHT_TYPE);

// ============================================================
// MQ135
// ============================================================

#define MQ135_PIN A0

// Adjust after checking your actual MQ135 readings
```

```
int airQualityLimit = 200;

// ========================================================
// BUZZER
// ========================================================

#define BUZZER_PIN D6

// ========================================================
// OLED
// ========================================================

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_RESET   -1
#define OLED_ADDRESS 0x3C

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  OLED_RESET
);
```

```
// ====================================================
// WIFI CLIENT
// ====================================================

WiFiClient client;

// ====================================================
// MQTT CLIENT
// ====================================================

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// ====================================================
// ADAFRUIT IO FEEDS
// ====================================================

Adafruit_MQTT_Publish temperatureFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
```

```
    AIO_USERNAME "/feeds/temperature"
  );

Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/humidity"
  );

Adafruit_MQTT_Publish airQualityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/air-quality"
  );

// ================================================
// CONNECT TO WIFI
// ================================================

void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
  {
    return;
  }
```

```
  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

// ====================================================
// CONNECT TO ADAFRUIT IO
// ====================================================

void connectMQTT()
```

```
{
  if (mqtt.connected())
  {
    return;
  }

  Serial.println("Connecting to Adafruit IO...");

  int8_t ret;

  while ((ret = mqtt.connect()) != 0)
  {
    Serial.print("MQTT Error: ");
    Serial.println(mqtt.connectErrorString(ret));

    mqtt.disconnect();

    delay(5000);
  }

  Serial.println("Adafruit IO Connected!");
}

// ================================================
// OLED STARTUP SCREEN
```

```
// ================================================

void showStartupScreen()
{
  display.clearDisplay();

  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);

  display.setCursor(0, 0);
  display.println("ENVIRONMENT MONITOR");

  display.setCursor(0, 18);
  display.println("DHT + MQ135");

  display.setCursor(0, 36);
  display.println("Connecting...");

  display.display();
}

// ================================================
// SETUP
// ================================================
```

```
void setup()
{
  Serial.begin(115200);

  // Start DHT
  dht.begin();

  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  // I2C
  Wire.begin(D2, D1);

  // OLED
  if (!display.begin(
        SSD1306_SWITCHCAPVCC,
        OLED_ADDRESS))
  {
    Serial.println("OLED NOT FOUND!");

    while (true)
    {
      delay(100);
    }
```

```
}

showStartupScreen();

delay(2000);

// WiFi
connectWiFi();

// Adafruit IO
connectMQTT();

// Ready screen
display.clearDisplay();

display.setTextColor(SSD1306_WHITE);
display.setTextSize(1);

display.setCursor(0, 0);
display.println("ENVIRONMENT MONITOR");

display.setCursor(0, 20);
display.println("System Ready");

display.setCursor(0, 40);
```

```
  display.println("WiFi + Cloud OK");

  display.display();

  Serial.println();
  Serial.println("======================================");
  Serial.println(" WIRELESS ENVIRONMENTAL MONITOR");
  Serial.println("======================================");
  Serial.println("DHT SENSOR  : READY");
  Serial.println("MQ135       : READY");
  Serial.println("OLED        : READY");
  Serial.println("BUZZER      : READY");
  Serial.println("WiFi        : CONNECTED");
  Serial.println("Adafruit IO : CONNECTED");
  Serial.println("======================================");
  Serial.println();
}

// ================================================
// MAIN LOOP
// ================================================

void loop()
{
  // ------------------------------------------------
```

```
// WIFI
// ------------------------------------------------

if (WiFi.status() != WL_CONNECTED)
{
  connectWiFi();
}

// ------------------------------------------------
// MQTT
// ------------------------------------------------

connectMQTT();

// ------------------------------------------------
// READ DHT
// ------------------------------------------------

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// ------------------------------------------------
// READ MQ135
// ------------------------------------------------
```

```
int airQuality = analogRead(MQ135_PIN);

// ------------------------------------------------
// CHECK DHT
// ------------------------------------------------

if (isnan(temperature) || isnan(humidity))
{
  Serial.println();
  Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");
  Serial.println("DHT SENSOR ERROR");
  Serial.println("Check DHT connections");
  Serial.println("!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!");

  digitalWrite(BUZZER_PIN, LOW);

  display.clearDisplay();

  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);

  display.setCursor(0, 0);
  display.println("ENVIRONMENT MONITOR");

  display.setCursor(0, 20);
```

```
  display.println("DHT SENSOR ERROR");

  display.setCursor(0, 40);
  display.println("Check connections");

  display.display();

  delay(2000);

  return;
}

// ------------------------------------------------
// AIR QUALITY STATUS
// ------------------------------------------------

bool airWarning = false;

if (airQuality >= airQualityLimit)
{
  airWarning = true;
}

// ------------------------------------------------
// BUZZER
```

```
// ------------------------------------------------

if (airWarning)
{
  digitalWrite(BUZZER_PIN, HIGH);
}
else
{
  digitalWrite(BUZZER_PIN, LOW);
}

// ------------------------------------------------
// SERIAL MONITOR
// ------------------------------------------------

Serial.println();
Serial.println("--------------------------------------");

Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" C");

Serial.print("Humidity    : ");
Serial.print(humidity, 1);
Serial.println(" %");
```

```
Serial.print("MQ135 Value : ");
Serial.println(airQuality);

if (airWarning)
{
  Serial.println("Air Quality : POOR");
  Serial.println("Buzzer      : ON");
}
else
{
  Serial.println("Air Quality : NORMAL");
  Serial.println("Buzzer      : OFF");
}

// ------------------------------------------------
// OLED DISPLAY
// ------------------------------------------------

display.clearDisplay();

display.setTextColor(SSD1306_WHITE);
display.setTextSize(1);

display.setCursor(0, 0);
```

```
display.println("ENVIRONMENT");


display.setCursor(0, 14);
display.print("Temp: ");
display.print(temperature, 1);
display.println(" C");


display.setCursor(0, 27);
display.print("Hum : ");
display.print(humidity, 1);
display.println(" %");


display.setCursor(0, 40);
display.print("Air : ");
display.println(airQuality);


display.setCursor(0, 53);


if (airWarning)
{
  display.println("AIR: POOR");
}
else
{
  display.println("AIR: NORMAL");
```

```
}

display.display();

// ------------------------------------------------
// SEND TEMPERATURE
// ------------------------------------------------

if (temperatureFeed.publish(temperature))
{
  Serial.println("Temperature : SENT");
}
else
{
  Serial.println("Temperature : FAILED");
}

// ------------------------------------------------
// SEND HUMIDITY
// ------------------------------------------------

if (humidityFeed.publish(humidity))
{
  Serial.println("Humidity    : SENT");
}
```

```
  else
  {
    Serial.println("Humidity    : FAILED");
  }

  // ------------------------------------------------
  // SEND AIR QUALITY
  // ------------------------------------------------

  if (airQualityFeed.publish(airQuality))
  {
    Serial.println("Air Quality : SENT");
  }
  else
  {
    Serial.println("Air Quality : FAILED");
  }

  Serial.println("-------------------------------------");
  Serial.println();

  // Update every 5 seconds
  delay(5000);
}
```

9. System Working

When the system is powered on, the ESP8266 initializes all connected sensors and the OLED display. It then attempts to connect to the available Wi-Fi network.

After initialization, the system continuously collects environmental measurements.

Temperature monitoring

The DHT11/DHT22 measures the surrounding temperature. If the temperature exceeds the configured threshold, the system changes its status to **WARNING** or **CRITICAL**.

Humidity monitoring

The humidity value is continuously measured. High or low humidity conditions can be identified according to the application's requirements.

Air-quality monitoring

The MQ-135 detects changes in its sensing environment. The ESP8266 processes its output and compares it with calibrated thresholds.

Light monitoring

An LDR can be used to detect changes in light intensity. This can be useful for smart-building and greenhouse applications.

10. Prototype Implementation

The prototype is constructed using an ESP8266 NodeMCU, environmental sensors, OLED display, buzzer, breadboard, and jumper wires.

Implementation procedure

Step 1: Place the ESP8266 NodeMCU on the breadboard.

Step 2: Connect the DHT11/DHT22 sensor to the ESP8266.

Step 3: Connect the MQ-135 air-quality sensor using suitable power and signal conditioning.

Step 4: Connect the LDR through a voltage-divider circuit.

Step 5: Connect the OLED using the I^2C interface.

Step 6: Connect the buzzer to the alarm output through a suitable driver if required.

Step 7: Upload the program using Arduino IDE.

Step 8: Connect the ESP8266 to Wi-Fi.

Step 9: Observe temperature, humidity, and air-quality readings on the OLED and Serial Monitor.

Step 10: Test different environmental conditions.

11. Results & Performance Analysis

The Wireless Environmental Parameter Monitoring System provides continuous environmental monitoring and wireless data transmission.

Expected test results

Parameter	Normal Condition	Abnormal Condition	System Response
Temperature	Normal range	High temperature	Warning/Alarm
Humidity	Acceptable range	Excessive humidity	Warning
Air Quality	Normal sensor value	High sensor value	Warning/Alarm
Light	Normal	Very low/high	Status update
Wi-Fi	Connected	Disconnected	Reconnection
Overall Status	Normal	Abnormal	Buzzer activation

Temperature performance

The DHT sensor continuously provides temperature readings. The controller can detect a rapid increase in temperature and generate an alert.

Humidity performance

Humidity is displayed in real time and can be monitored remotely. This is useful in laboratories, warehouses, greenhouses, and indoor environments where humidity control is important.

Air-quality performance

The MQ-135 provides a useful low-cost indication of changes in air quality. It can be used to identify unusual changes in the environment, although proper calibration is necessary for quantitative air-quality measurement.

12. Bill of Materials

S.No.	Component	Quantity	Approx. Cost
1	ESP8266 NodeMCU	1	₹300–₹500
2	DHT11/DHT22	1	₹80–₹250
3	MQ-135 Air Quality Sensor	1	₹100–₹200
4	LDR	1	₹5–₹20
5	OLED 0.96-inch I^2C	1	₹150–₹300
6	Buzzer	1	₹10–₹30
7	ADS1115 ADC	1	₹150–₹300
8	Breadboard	1	₹80–₹150
9	Jumper Wires	1 set	₹50–₹150
10	Resistors	1 set	₹20–₹50
11	USB Cable	1	₹50–₹150
12	5 V USB Power Supply	1	₹100–₹250

Estimated prototype cost

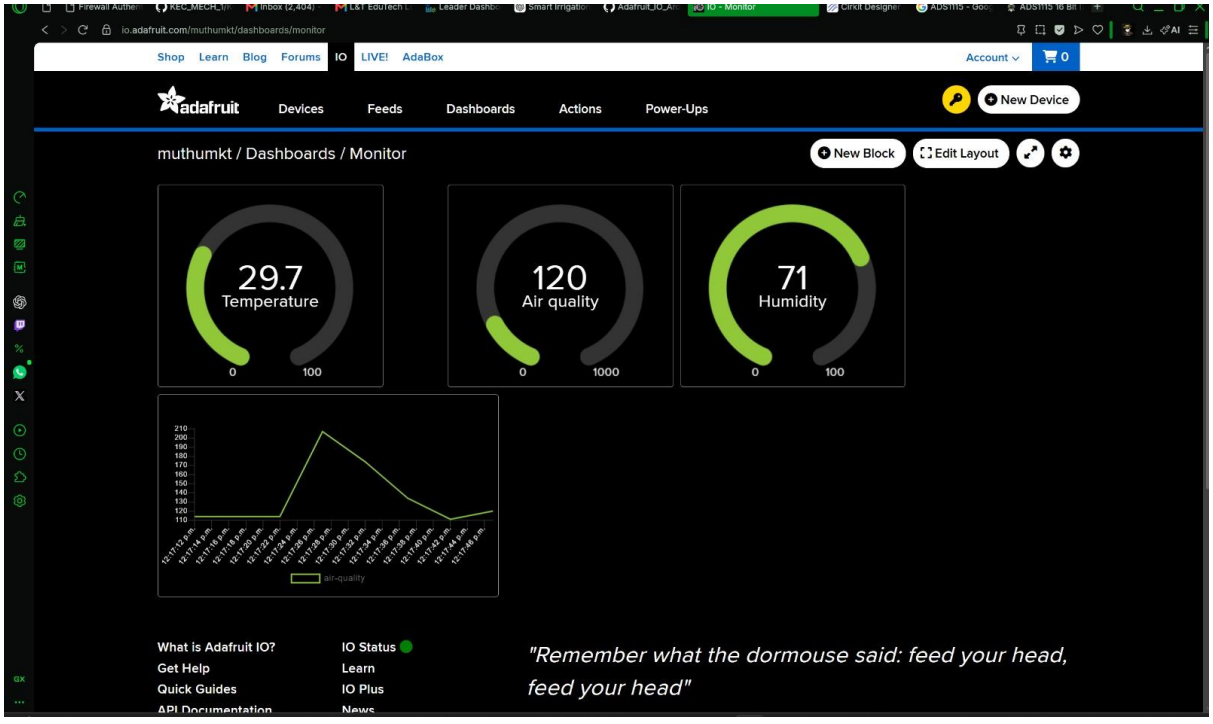
Approximately ₹1,095–₹2,350, depending on the selected sensors and components.

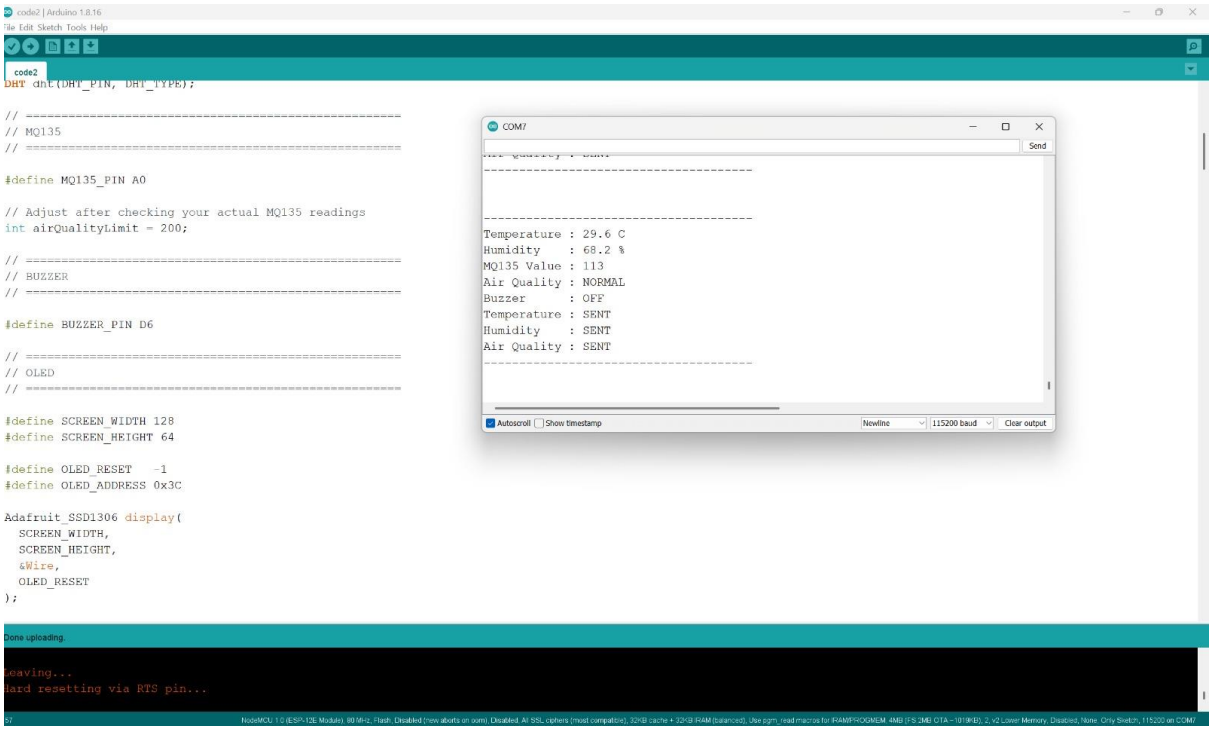
Conclusion

The **Wireless Environmental Parameter Monitoring System** provides an efficient and economical method for monitoring environmental conditions in real time. The ESP8266 NodeMCU collects information from environmental sensors, processes the data, displays the readings locally, generates alerts for abnormal conditions, and provides wireless connectivity for remote monitoring.

The system can significantly reduce the need for manual environmental measurements and can be expanded into a complete IoT monitoring platform by adding cloud data logging, mobile notifications, historical graphs, multiple sensor nodes, and automated control systems.

OUTPUT:




code2 | Arduino 1.8.16
File Edit Sketch Tools Help
code2
DHT dht(DHT_PIN, DHT_TYPE);
// MQ135
#define MQ135_PIN A0
int airQualityLimit = 200;
// BUZZER
#define BUZZER_PIN D6
// OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDRESS 0x3C
Adafruit_SSD1306 display(
SCREEN_WIDTH,
SCREEN_HEIGHT,
&Wire,
OLED_RESET
);
COM7
Send
Temperature : 29.6 C
Humidity : 68.2 %
MQ135 Value : 113
Air Quality : NORMAL
Buzzer : OFF
Temperature : SENT
Humidity : SENT
Air Quality : SENT
Autoscroll
Show timestamp
Newline
115200 baud
Clear output
Done uploading.
leaving...
Hard resetting via RTS pin...